



Features

App Fees

Copy page



Add App Fees on any route and collect in stablecoins

What are App Fees?

App fees are additional fees set by third-party developers on top of existing fees. These fees are specified when generating a quote. To improve the gas costs of fee setting, App fees are not paid out in real-time. Rather, App Fees accrue in an offchain base *USDC*, balance and can be withdrawn as desired at your convenience.

Setting App Fees

App Fees are measured in basis points (bps) based on the input value of an order. When setting an app fee, an EVM-based claim address must be provided, which will be used for withdrawing accumulated fees. A single quote can include multiple App fees. You can accrue fees by including **AppFees** in the Quote API:

```
"appFees": [  
  {  
    "recipient": "0x999999cf1046e68e36E1aA2E0E07105eDDD1f08E", //EVM Claim Address  
    "fee": "100" //App Fee (in BPS of input token)  
  }  
]
```

Additional Considerations

- On same-chain operations where the solver holds neither the input or output currency, **RELAY** an extra swap is added within the transaction to swap from the input currency into a currency accepted by the solver for App Fees
- For this to occur, the fee to be taken needs to be greater than \$0.025. This is to prevent the situation where the action of taking the app fee will consume more in gas than the fee itself.
- App Fee Recipients *must* be an EVM compatible wallet address, even for non-EVM chains.
- App fees *can* be an array, for setting multiple fees at once
- App fees work the same way with deposit address flows as with regular quote requests.

⚠ If you include the `appFees` parameter on an unsupported transaction type (such as sends, transfers, or gasless execution without a prior quote), the API will not return an error — but no fees will be collected. Always verify fee collection is working using the [monitoring endpoints](#) described below.

Supported Transaction Types

App fees are supported on the following transaction types:

Transaction Type	Supported
Swaps (cross-chain)	✓ All chain combinations (EVM, SVM, SUI, BTC)
Swaps (same-chain)	✓ See chain-specific notes below
Bridges	✓
Calls	✓
Wraps / Unwraps	✓
Sends / Transfers	✗
Gasless swaps	✓ Fees from original quote are respected
Gasless execution (no prior quote)	✗ Same-chain only — e.g. 7702 wallet upgrades

Chain-Specific Notes

RELAY ✱

EVM (same-chain): Fully supported. Fees can be collected from the input currency, the output currency, or via a parallel swap to USDC.

SVM (same-chain): Supported with input-currency collection only. Output-currency fee collection is not available — this applies to all output currencies, not just SOL. If the input currency is not a solver-held currency, the fee is collected via a secondary swap into USDC.

SUI (same-chain): Supported with input-currency collection only. No output-currency collection or parallel swap fallback. App fees will only be collected when the input is a solver-held currency.

Cross-chain (all combinations): Fully supported. Same-chain limitations do not apply to cross-chain swaps — fees are calculated based on the solver's deposit currency on the origin chain.

Verifying Fee Collection

You can verify that app fees are being collected correctly using the following endpoints:

Per-transaction: Use `/requests/v2` which returns both `appFees` (the fee quoted to the user) and `paidAppFees` (the fee actually collected). If `appFees` is present but `paidAppFees` is empty or zero, the fee was not collected for that transaction.

Accumulated balance: Use `/app-fees/{address}/balances` to check your total accumulated app fee balance available for withdrawal.

Withdrawing App Balance

App balance accrue in USDC from app fees, and can be claimed on any chain. The balance is free to claim on Base, and Relay app balance apply to claim on any other chain or in any other currency.

The easiest way to claim your App Balance is to use the [Relay App](#). The Relay App Balance page also supports withdrawing via a multisig. Here's how you can withdraw via the API:

Get App Balance

To check your accrued App Balance, use the App fee balances API. Here's an example curl command and response. The response includes the `balances.currency` field reflecting your off-chain balance for each currency accumulated.

cURL Response



```
curl --location 'https://api.relay.link/app-fees/0x03508bb71268bba25ecacc8f620e01
```

To learn more about this api, see the [Get App Fee Balance](#) API reference.

How to withdraw your off-chain balance?

To withdraw your app fees, use the app fees claim API. If using the API, the response will guide you through signing and submitting a message to the `/execute/permit` endpoint to verify ownership of your claim address. The SDK automates this process for you.

cURL SDK Response



```
curl --request POST \  
  --url https://api.relay.link/app-fees/0x03508bb71268bba25ecacc8f620e01866650532 \  
  --header 'Content-Type: application/json' \  
  --data '{  
    "chainId": 8453,  
    "currency": "0x0000000000000000000000000000000000000000000000000000000000000000",  
    "recipient": "0x03508bb71268bba25ecacc8f620e01866650532c"  
  }'
```

If you're using the API directly, you can then send a request to `/execute/permit` with the `requestId` and `signature` to claim your fees. An example request would be:



```
curl -X POST \
  https://api.relay.link/execute/permits?signature=${signature} \
  -H 'Content-Type: application/json' \
  -d '{
    "kind": "request",
    "requestId": "0xa344c6123e3da9fc3f9c81edeab0b2eb39f2ea80ba54a6e0ad00123dc1806
  }'
```

Once done, you can check your balance again to verify the withdrawal. To learn more about this api, see the [Claim App Fee Balance](#) API reference.

Withdrawing via a smart contract

In some cases you may want to accrue fees into a smart contract instead of an EOA. This can easily be done by setting the app fees recipient. The challenge is in claiming the fees to the contract. The claim API requires a signature to claim app fees but contracts cannot sign messages. To solve this, the API supports [EIP-1271](#) which allows a smart contract to validate a signature using arbitrary logic. Once implemented our backend will check if the contract implements `isValidSignature` and use that method to verify the signature.

Here is an example of how this might work in practice:

- Wallet A (owner by contract owner), signs the message
- Then our backend gets the posted signature and calls the contract's `isValidSignature` method to verify the signature
- If the signature is valid, the funds get sent to the recipient. Let's say in this case the contract's `isValidSignature` method returns true if the signature was generated by wallet A.

Was this page helpful?



 Yes

 No

>
< **Call Execution**

Fee Sponsorship >

Powered by **mintlify**